

Chapitre 5: Gestion de la pile et du Tas

Cours 7: Segmentation de la mémoire en C

Plan

1) Rappels sur les composants d'un processeur

- Les registres

2) Segmentation de la mémoire

- Le segment *text*
- Le segment *données*
- Le segment TAS
- Le segment Pile

Plan

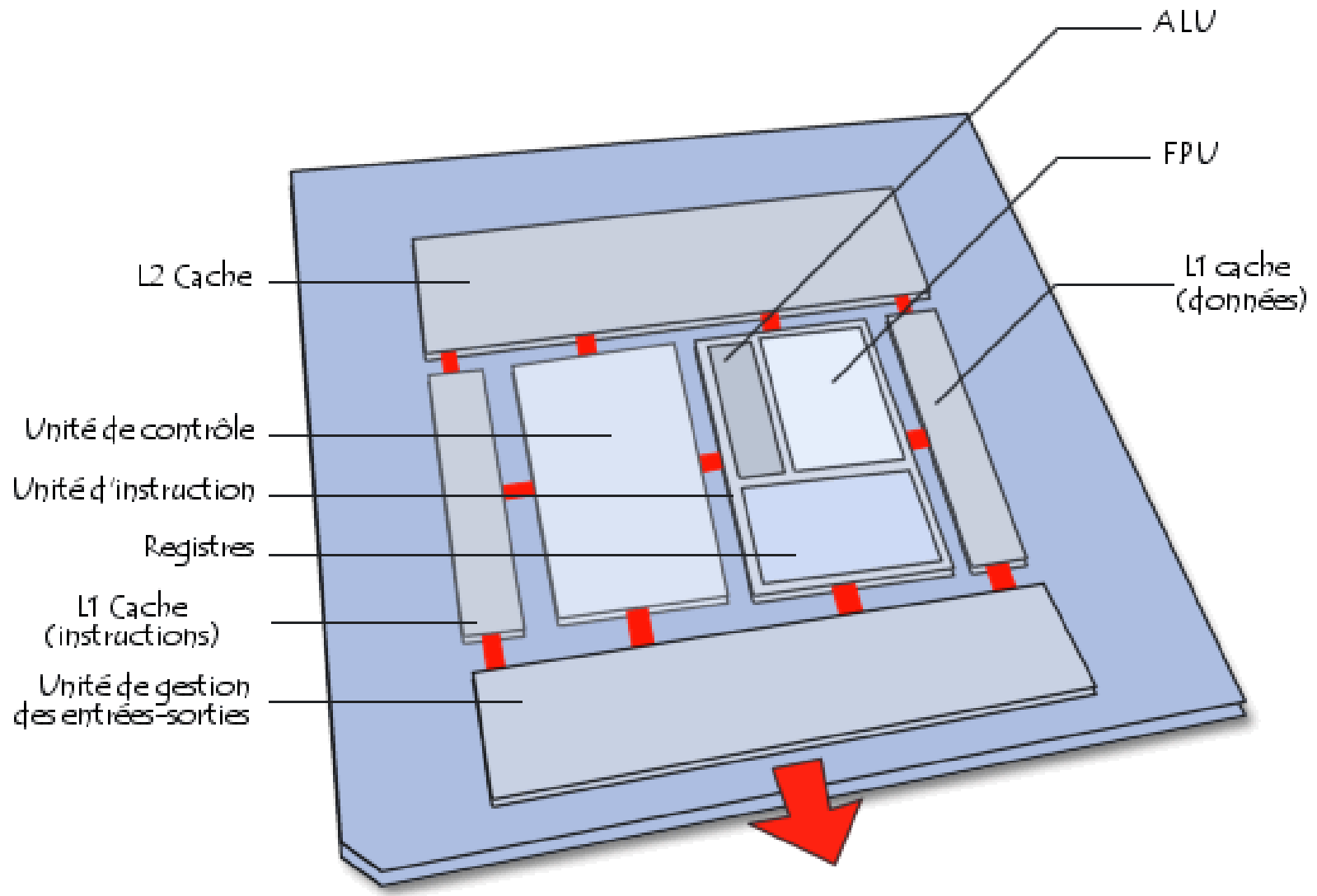
1) Rappels sur les composants d'un processeur

- Les registres

2) Segmentation de la mémoire

- Le segment *text*
- Le segment *données*
- Le segment TAS
- Le segment Pile

Rappel sur les composant d'un Processeur



Plan

1) Rappels sur les composants d'un processeur

- Les registres

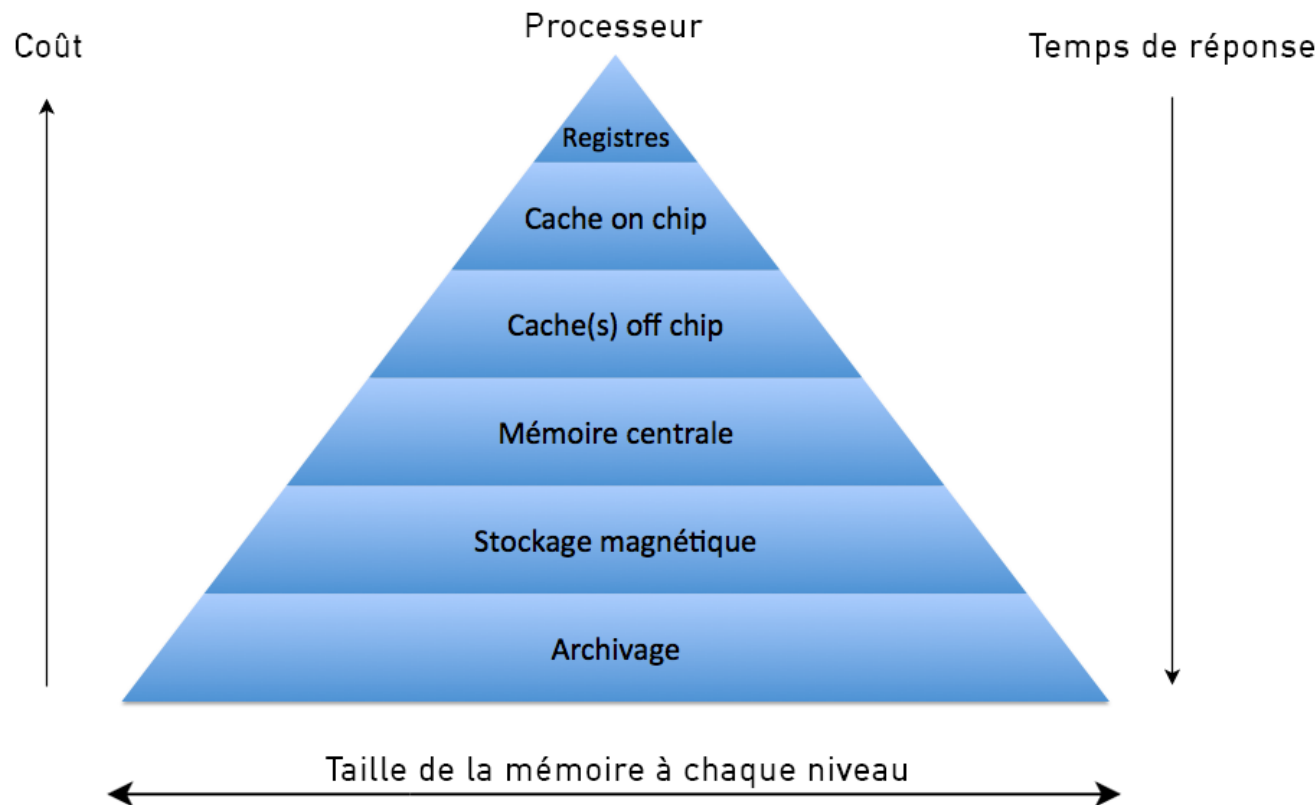
2) Segmentation de la mémoire

- Le segment *text*
- Le segment *données*
- Le segment TAS
- Le segment Pile

Les registres

- Les registres sont des emplacements se trouvant à l'intérieur du processeur.

- Or dans un ordinateur, les emplacements mémoire les plus proches du processeur sont les plus rapides à y accéder, mais également les plus chers.



- Ainsi, plus on s'éloigne du processeur, plus les accès sont longs, mais les coûts sont faibles.

Les registres

- Le processeur x86 de 32 bits possède (logiquement) 8 registres généraux (EAX, EBX, ECX, EDX, ESI, EDI, EBP, ESP)
- Pour les processeurs 64 bits, il y a 16 registres logiques.

31	15	7	0
EAX	AX (AH)	AX (AL)	
EBX	BX (BH)	BX (BL)	
ECX	CX (CH)	CX (CL)	
EDX	DX (DH)	DX (DL)	
ESI	SI		
EDI	DI		
EBP	BP		
ESP	SP		
EFLAGS	FLAGS		
EIP	IP		

Les registres

On distingue deux groupes:

Les 4 E?X définis comme suit:

- E**A**X = **A**ccumulateur,
- E**B**X = **B**ase,
- E**C**X = **C**ompteur, et
- E**D**X = **D**onnées.

Ils ont pour rôle de stocker des données temporaires pour le processeur lorsqu'il exécute un programme.

31	15	7	0
EAX	AX (AH)	AX (AL)	
EBX	BX (BH)	BX (BL)	
ECX	CX (CH)	CX (CL)	
EDX	DX (DH)	DX (DL)	
ESI	SI		
EDI	DI		
EBP	BP		
ESP	SP		
EFLAGS	FLAGS		
EIP	IP		

Les registres

Les 4 autres registres sont utilisés en tant que pointeurs et index:

- **ESI** = **I**ndex de **S**ource,
- **EDI** = **I**ndex de **D**estination,
- **EBP** = **P**ointeur de **B**ase,
- **ESP** = **P**ointeur de Pile (**S**tack).

Par exemple, ESP et EBP stockent des adresses mémoires pour délimiter le stack frame courant.

31	15	7	0
EAX	AX (AH)	AX (AL)	
EBX	BX (BH)	BX (BL)	
ECX	CX (CH)	CX (CL)	
EDX	DX (DH)	DX (DL)	
ESI	SI		
EDI	DI		
EBP	BP		
ESP	SP		
EFLAGS	FLAGS		
EIP	IP		

Les registres

Nous avons également deux autres registres, un peu plus spéciaux :

- Le registre **EFLAGS** qui contient des indicateurs, des interrupteurs, des drapeaux (*flags*) essentiellement utilisés pour des comparaisons.
- Enfin, Le registre **EIP** est appelé **P**ointeur d'**I**nstruction. Il contient l'adresse de la prochaine instruction que le processeur doit exécuter.

31	15	7	0
EAX	AX (AH)	AX (AL)	
EBX	BX (BH)	BX (BL)	
ECX	CX (CH)	CX (CL)	
EDX	DX (DH)	DX (DL)	
ESI	SI		
EDI	DI		
EBP	BP		
ESP	SP		
EFLAGS	FLAGS		
EIP	IP		

Plan

1) Rappels sur les composants d'un processeur

- Les registres

2) Segmentation de la mémoire

- Le segment *text*
- Le segment *données*
- Le segment TAS
- Le segment Pile

Segmentation de la mémoire

- Un **algorithme** est une suite d'instructions permettant de résoudre un problème donnée,
- Cet algorithme est traduit dans un langage de programmation pour générer ce qu'on appelle un **programme**,
- Ce programme est alors compilé, ensuite chargé en mémoire pour s'exécuter et devient par conséquent un **processus**,
- Ainsi, nous allons voir ici comment se présente un processus dans la mémoire, on parle de **segmentation** de la mémoire.

Segmentation de la mémoire

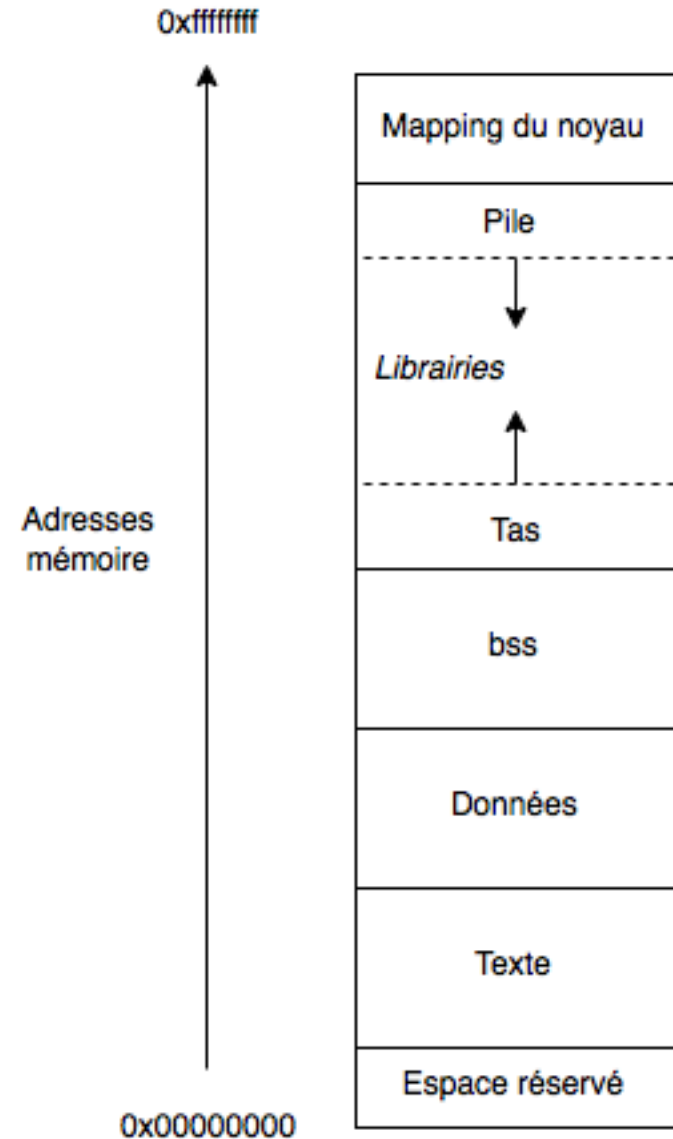
On retrouve les 3 sections suivantes :

- Texte (*.text*)
- Données (*.data*)
- bss (*.bss*)

Et les 2 zones mémoire suivantes :

- Tas (*heap*)
- Pile (*stack*)

Chacune de ces zones représente une partie de la mémoire allouée au processus en question.



Plan

1) Rappels sur les composants d'un processeur

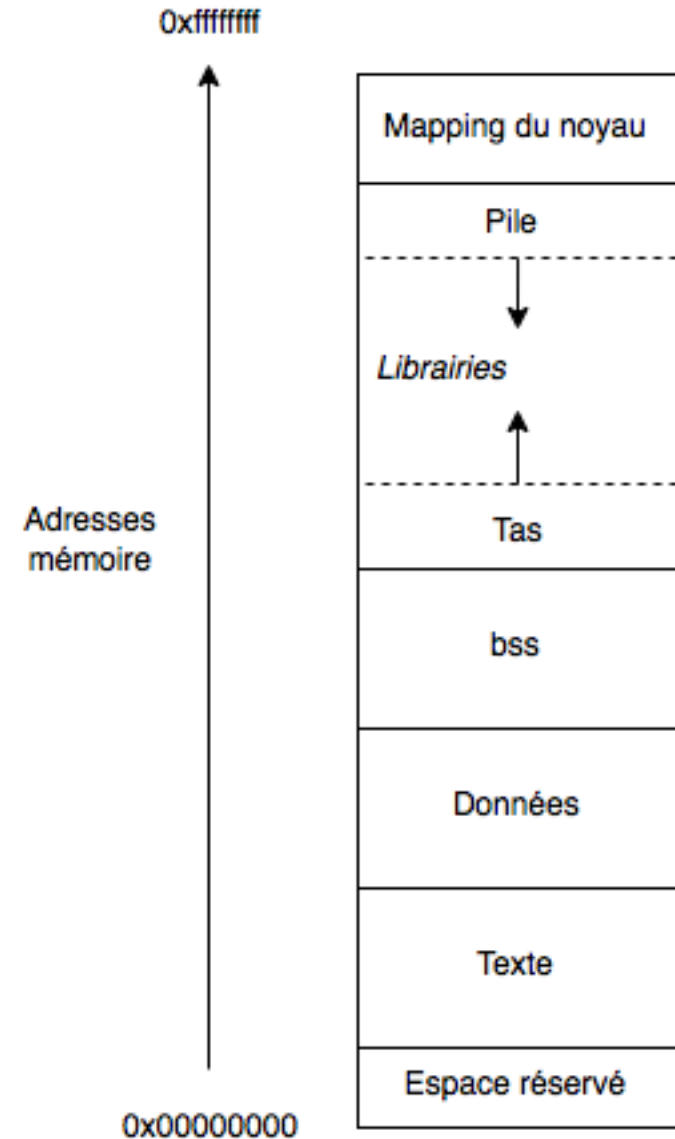
- Les registres

2) Segmentation de la mémoire

- **Le segment *text***
- Le segment *données*
- Le segment TAS
- Le segment Pile

Le segment text

- La **section texte** (*.text*) contient le code du programme (instructions en langage machine). Elle sert seulement à stocker du code, pas de variables.
- C'est une section en lecture seule, une fois qu'elle a été définie, elle est **immuable**, donc de taille fixe.



Plan

1) Rappels sur les composants d'un processeur

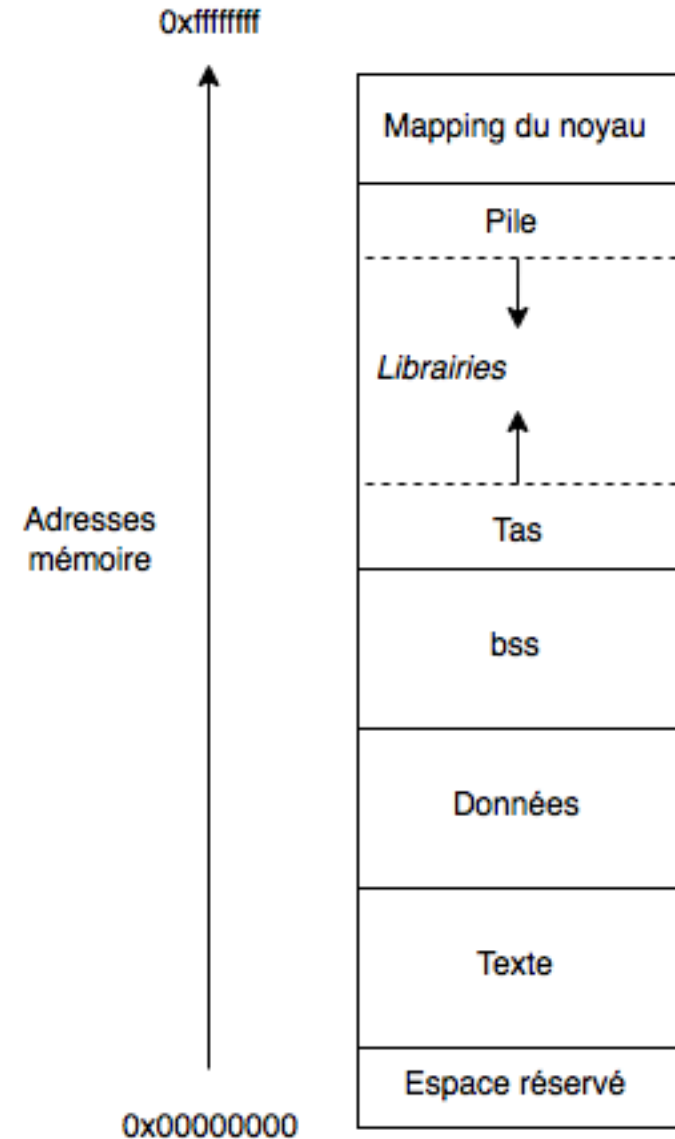
- Les registres

2) Segmentation de la mémoire

- Le segment *text*
- **Le segment *données***
- Le segment TAS
- Le segment Pile

Le segment données

- La section de **données** (*data*) stockent les variables globales et statiques du programme à condition qu'elles soient **initialisées**.
- Si elles ne sont pas initialisées, elles seront stockées dans la section **bss**.
- Les sections *data* et *bss* sont des zones mémoires de taille fixe.



Plan

1) Rappels sur les composants d'un processeur

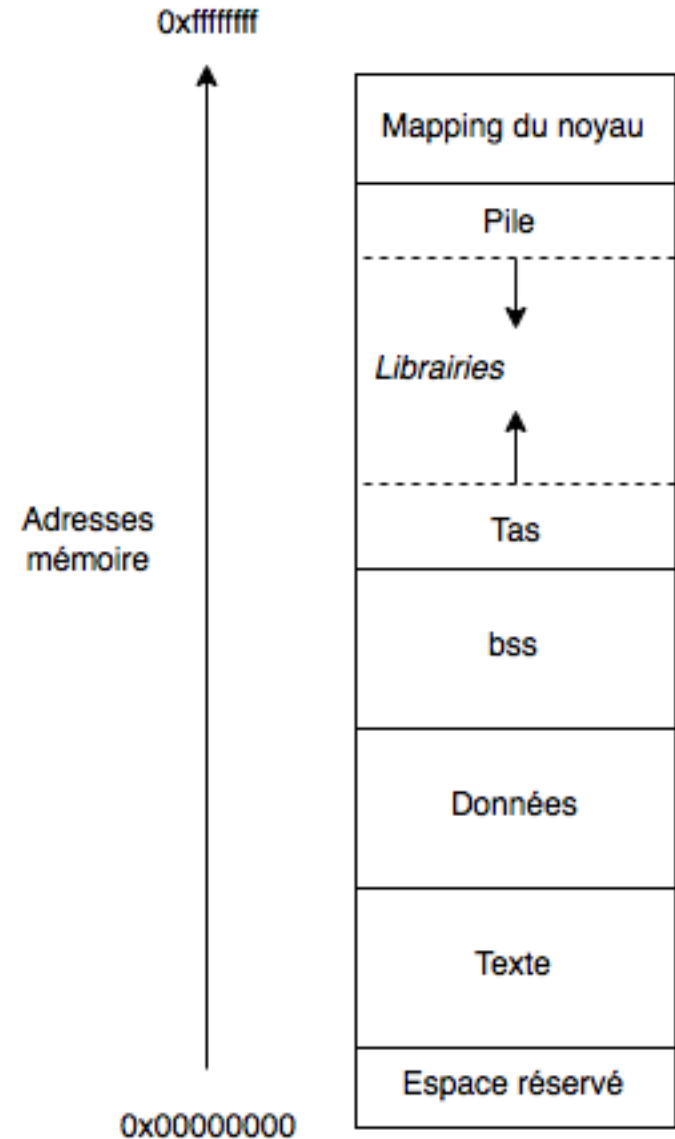
- Les registres

2) Segmentation de la mémoire

- Le segment *text*
- Le segment *données*
- **Le segment TAS**
- Le segment Pile

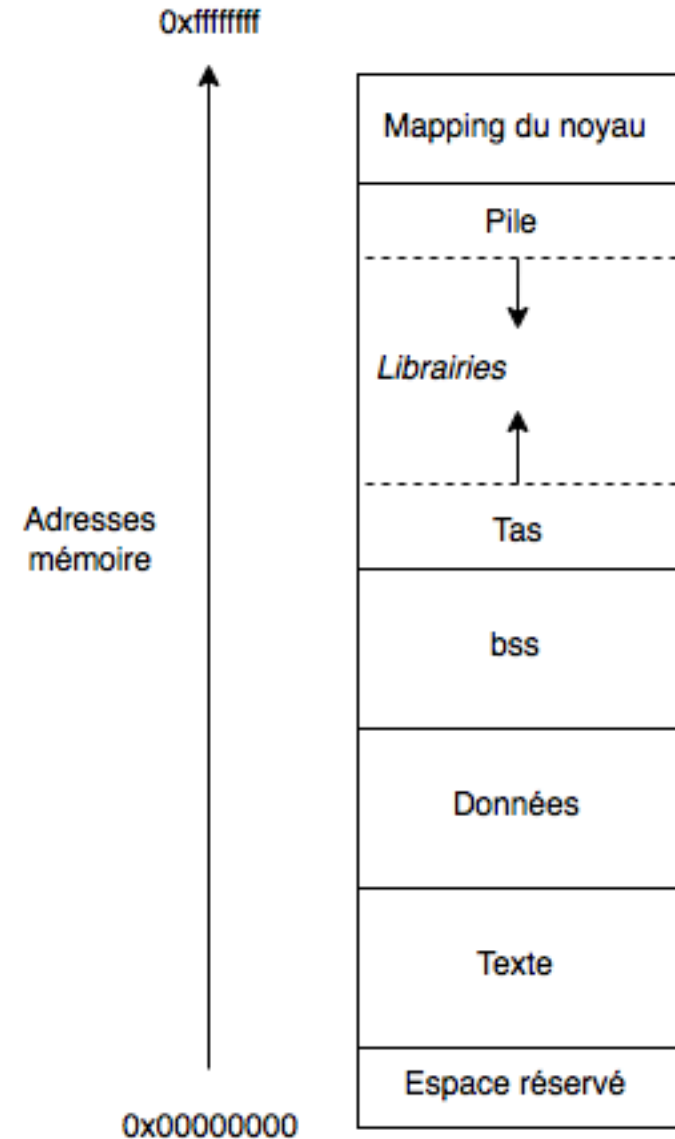
Le segment TAS

- Le **tas** (*heap*) est une zone dans laquelle sont écrites les zones mémoires allouées dynamiquement (*malloc()* ou *calloc()*).
- Cette zone mémoire n'a pas de taille fixe. Elle augmente et diminue en fonction des demandes du programmeur, qui peut réserver ou supprimer des blocs via des algorithmes d'allocation ou de libération.
- Plus la taille du tas augmente, plus les adresses mémoires augmentent, et s'approchent des adresses mémoires de la pile.



Le segment TAS

- La taille des variables dans le tas n'est pas limitée (sauf limite physique de la mémoire), contrairement à la pile.
- Par ailleurs, les variables stockées dans le tas sont accessibles partout dans le programme, par l'intermédiaire des **pointeurs**.



Plan

1) Rappels sur les composants d'un processeur

- Les registres

2) Segmentation de la mémoire

- Le segment *text*
- Le segment *données*
- Le segment TAS
- **Le segment Pile**

Le segment Pile

- La **pile** (*stack*) possède également une taille variable, mais plus sa taille augmente, plus les adresses mémoires diminuent, en s'approchant du haut du tas.
- C'est ici qu'on retrouve les variables locales des fonctions et d'autres informations.

